

#####

APRENDIZAJE NO SUPERVISADO EN KERAS.

MODULO 4

#####

== INTRODUCCION ==

- EL APRENDIZAJE NO SUPERVISADO ES UN TIPO DE MACHINE LEARNING.
- EL ALGORITMO SE USA PARA ENCONTRAR PATRONES EN LOS DATOS SIN ETIQUETAS NI RESULTADOS PREDEFINIDOS

└─> ● EN EL APRENDIZAJE SUPERVISADO TENEMOS UNA VARIABLE OBJETIVO QUE PREDECIR.

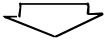
- NO SUPERVISADO: COMPRENDER LA ESTRUCTURA SUBYACENTE DE LOS DATOS.

≡ CLASIFICACIÓN ≡

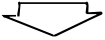
• TRES TIPOS:

CLUSTERING

AGRUPAR LOS
PUNTOS DE DATOS
EN CLUSTERS.



LOS PUNTOS DE
CADA CLUSTER
MÁS SIMILARES
ENTRE SÍ.

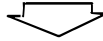


EJEMPLOS:

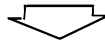
- K-MEAN CLUSTERING
- AGRUPAMIENTO GEOGRÁFICO.

ASSOCIATION

ENCENTRAR
RELACIONES
ENTRE VARIABLES
Y GRUPOS CONJUNTOS
DE DATOS.



SE USA NORMALMENTE
EN EL ANÁLISIS DE LA
CESTA DE LA COMPRA
PARA IDENTIFICAR
PRODUCTOS QUE SON
FRECUENTEMENTE COMPRADOS
CON LOS TRANSACCIONES.

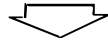


EJEMPLOS:

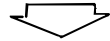
- Algoritmo: A PRIORI
- Algoritmo: ECLAT

DIMENSIONALITY REDUCTION

REDUCIR EL N° DE
VARIABLES ALGUNAS



OBTENCIÓN DE
VARIABLES
PRINCIPALES



EJEMPLOS:

- PCA: NÚMEROS DE
COMPONENTES
PRINCIPALES

T-SNE: REPRESENTACIÓN DE
VECORES ESTOCÁSTICOS
DISTRIBUIDOS.

≡ AUTOENCODERS ≡

* SON UN TIPO DE RED NEURONAL QUE SE
UTILIZA PARA APRENDER REPRESENTACIONES
EFICIENTES DE DATOS

* CON EL FIN DE REDUCIR LA DIMENSIONALIDAD
O APRENDER CARACTERÍSTICAS.

* DOS PARTES:

- ENCODER: COMPRIME LA ENTRADA EN UNA
REPRESENTACIÓN DE ESPACIO LATENTE

- DECODER: RECONSTRUYE LA ENTRADA DE LA
REPRESENTACIÓN DEL ESPACIO LATENTE.

● IMPORTANTE:

- EL AUTOENCODER ESTÉ ENTRENADO PARA
MINIMIZAR LA DIFERENCIA ENTRE LA
ENTRADA Y LA SALIDA RECONSTRUIDA

- ESTO OBLIGA A LA RED A APRENDER
REPRESENTACIONES SIGNIFICATIVAS DE LOS
DATOS.

AUTOENCODER EN KERAS

Define the encoder

input-layer = Input(shape=(784,))

encoded = Dense(64, activation='relu')(input-layer)

Define the decoder

decoded = Dense(784, activation='sigmoid')(encoded)

Combine the encoder and decoder into an autoencoder model

autoencoder = Model(input-layer, decoded)

Summary the model

autoencoder.summary()

* TRAINING THE AUTOENCODER *

Load the MNIST dataset

```
(x_train, _), (x_test, _) = tf.keras.datasets.mnist.load_data()
```

Normalize data

```
x_train = x_train.astype('float32') / 255.
```

```
x_test = x_test.astype('float32') / 255.
```

```
x_train = x_train.reshape((len(x_train), 784))
```

```
x_test = x_test.reshape((len(x_test), 784))
```

Train the autoencoder

```
autoencoder.fit(x_train, x_train,
```

```
    epochs=50,
```

```
    batch_size=256,
```

```
    shuffle=True,
```

```
    validation_data=(x_test, x_test))
```

** MNIST = Institut National de Etudes

≡ REDES GENERATIVAS DE CONTRAFACCIÓN ≡

GANs

* SON UNA CLASE DE REDES NEURONALES DISEÑADAS POR : JOHN GOODFELLOW EN 2014.

* CONSISTEN EN DOS REDES:

- GENERADORA

- DISCRIMINADORA



COMPLETO SU TAREA SI
SU UN JUEGO DE
SUMA CERO.

* RED GENERADORA : GENERA NUEVAS INSTANCIAS DE DATOS QUE SE PAREZCAN A LOS DATOS DE ENTRENAMIENTO.

* RED DISCRIMINADORA : EVALÚA LA AUTENTICIDAD DE LOS DATOS GENERADOS.

* EL GENERADOR INTENTA ENGañAR AL DISCRIMINADOR MIENTRAS EL DISCRIMINADOR TRATA DE DISTINGUIR ENTRE DATOS REALES Y FALSOS.

* ESTE PROCESO CONTRADICTORIO LLEVA AL GENERADOR A PRODUCIR DATOS CADA VEZ MÁS REALISTAS.

* IMPLEMENTING SIMPLE GAN *

Define the generator model

```
def build_generator():  
    model = tf.keras.Sequential()  
    model.add(Dense(784, input_dim=100))  
    model.add(LeakyRelu(alpha=0.01))  
    model.add(Dense(784, activation='tanh'))  
    return model
```

Define the discriminator model

```
def build_discriminator():  
    model = tf.keras.Sequential()  
    model.add(Dense(128, input_shape=(784,)))  
    model.add(LeakyRelu(alpha=0.01))  
    model.add(Dense(1, activation='sigmoid'))  
    return model
```

Build & compile the discriminator

discriminator = build_discriminator()

discriminator.compile(optimizer='adam', loss=
'binary_crossentropy', metrics=['accuracy'])

Build the generator

generator = build_generator()

Create the GAN by combining the generator
and discriminator

discriminator.trainable = ~~True~~

gen_input = Input(shape=(100,))

generated_image = generator(gen_input)

gen_output = discriminator(generated_image)

gan = Model(gen_input, gen_output)

Compile the GAN

gan.compile(optimizer='adam', loss='binary_crossentropy')

Training the GAN

```
def train_gan(gan, generator, discriminator,  
              x_train, epochs = 400, batch_size = 128):
```

```
    # Loop through epochs
```

```
    for epochs in range(epochs):
```

```
        # Generate random noise as input for the generator
```

```
        noise = np.random.normal(0, 1, (batch_size, 100))
```

```
        generate_images = generator.predict(noise)
```

```
    # Get a random set of real images
```

```
    idx = np.random.randint(0, x_train.shape[0],
```

```
                             batch_size)
```

```
    real_images = x_train[idx]
```

```
    # Labels for real and fake images
```

```
    real_labels = np.ones((batch_size, 1))
```

```
    fake_labels = np.zeros((batch_size, 1))
```

```
# Train the discriminator on real and fake images separately
d_loss_real = discriminator.train_on_batch(real_images, real_labels)
d_loss_fake = discriminator.train_on_batch(generator_images, fake_labels)
```

```
# Calculate the average loss for the discriminator
```

```
d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)
```

```
# Generate new noise and train the generator through the GAN model (note: we train the generator via the GAN model, where the discriminator's weights are frozen)
```

```
noise = np.random.normal(0, 1, (batch_size, 100))
g_loss = generator.train_on_batch(noise, real_labels, return_dict=True)
```

```
# Print the progress every 10 epochs
```

```
if epoch % 10 == 0:
```

```
    print(f"Epoch {epoch} - Discriminator Loss: {d_loss[0]}, Generator Loss: {g_loss['loss']}")
```

```
    return d_loss, g_loss
```


#####

CREACIÓN AUTOCODIFICADORES EN KERAS

MODULO 4

#####

== INTRODUCCIÓN ==

* LOS AUTOENCODERES SON UN TIPO DE RED NEURONAL QUE SE USA PARA TAREAS DE APRENDIZAJE NO SUPERVISADO.

* USADOS EN:

- REDUCCIÓN DE DIMENSIONES
- ELIMINACIÓN DE RUIDO
- APRENDIZAJE DE FUNCIONES

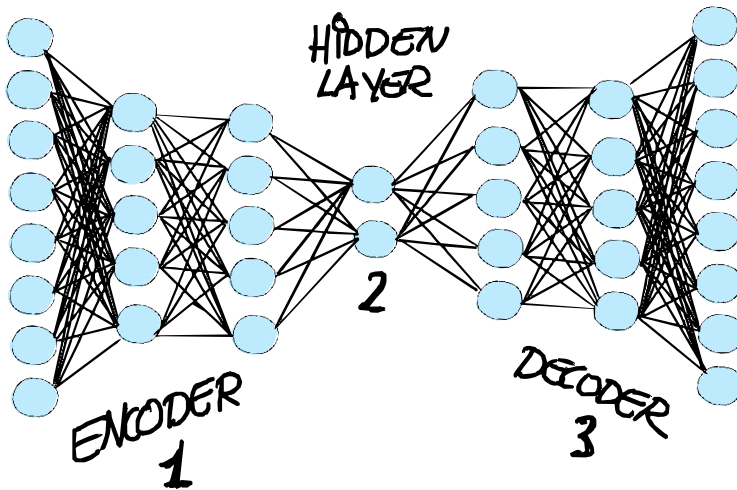
== ¿QUE ES UN AUTOENCODER? ==

* UN CODIFICADOR AUTOMÁTICO ES UN TIPO DE RED NEURONAL QUE SE UTILIZA PARA APRENDER:

- REPRESENTACIONES EFICIENTES DE DATOS

└─> CON EL OBJETIVO DE REDUCIR LA
DIMENSIONALIDAD.

- CONSTA DE UN CODIFICADOR Y UN
DECODIFICADOR



≡ ARQUITECTURA BÁSICA ≡

- ENCODER: COMPRIME LOS DATOS DE ENTRADA EN UNA REPRESENTACIÓN MÁS PEQUEÑA
- BOTTLENECK: BLOQUEA LA REPRESENTACIÓN COMPRIMIDA QUE CONTIENE LAS CARACTERÍSTICAS MÁS IMPORTANTES.

* DECODER: RECONSTRUYE LOS DATOS DE ENTRADA A PARTIR DE LA REPRESENTACIÓN COMPACTA

* OBJETIVO: MINIMIZAR LA DIFERENCIA ENTRE LOS DATOS DE ENTRADA Y LOS DATOS RECONSTRUIDOS.

== TIPOS DE ENCODERS ==

* BASIC AUTOENCODERS:

- ESTRUCTURA SIMPLE CON UNA SOLA CAPA OCULTA TANTO EN EL CODIFICADO COMO EN EL DECODIFICADO.

* VAEs (VARIATIONAL AUTOENCODERS):

- INTRODUCEN ELEMENTOS PROBABILÍSTICOS Y SE UTILIZAN PARA GENERAR NUEVAS MUESTRAS DE DATOS.

* CONVOLUTIONAL AUTOENCODERS:

- UTILIZAN CAPAS CONVOLUCIONALES Y SON MUY EFICACES PARA LOS DATOS DE IMAGEN.

* AUTOENCODER KERAS *

Encoder

```
input_layer = Input(shape=(784,))  
encoder = Dense(64, activation='relu')(input_layer)
```

Bottleneck

```
bottleneck = Dense(32, activation='relu')(encoder)
```

Decoder

```
decoded = Dense(64, activation='relu')(bottleneck)  
output_layer = Dense(784, activation='sigmoid')(decoded)
```

Autoencoder Model

```
autoencoder = Model(input_layer, output_layer)
```

Compile the model

```
autoencoder.compile(optimizer='adam', loss=  
    'binary_crossentropy')
```

Summary of the model

```
autoencoder.summary()
```

Training the model

Load and preprocess the data set

```
(x_train, _), (x_test, _) = mnist.load_data()
x_train = x_train.astype('float32') / 255.
x_test = x_test.astype('float32') / 255.
x_train = x_train.reshape((len(x_train), np.prod(
    x_train.shape[1:]))
x_test = x_test.reshape((len(x_test), np.prod(
    x_test.shape[1:])))
```

Train the model

```
autoencoder.fit(x_train, x_train, epochs=50,
    batch_size=256, shuffle=True,
    validation_data=(x_test, x_test))
```

Fine-Tuning the autoencoder

Unfreeze the top layers of the encoder

for layer in autoencoder.layers[-4:]:

layer.trainable = True

Compile the model

autoencoder.compile(optimizer='adam', loss='binary-crossentropy')

Train the model

autoencoder.fit(x_train, x_train, epochs=10,
batch_size=256, shuffle=True,
validation_data=(x_test, x_test))

#####

LABORATÓRIO VII

MODULO 4

#####

CONSTRUÇÃO DE AUTO ENCODERS



#####

#####

MODELOS DE DIFUSIÓN

MODULO 4

#####

INTRODUCCIÓN

* LOS MODELOS DE DIFUSIÓN SON UNA CLASE DE MODELOS GENERATIVOS QUE HAN GANADO POPULARIDAD POR SU CAPACIDAD DE PRODUCIR DATOS SINTÉTICOS DE ALTA CALIDAD:

- PRODUCEN DATOS SINTÉTICOS DE GRAN CALIDAD
- APLICACIONES DE GENERACIÓN Y MEJORA DE IMÁGENES

* SON MODELOS PROBABILÍSTICOS QUE GENERAN DATOS REFINANDO ITERATIVAMENTE UNA MUESTRA INICIAL RUIDOSA.

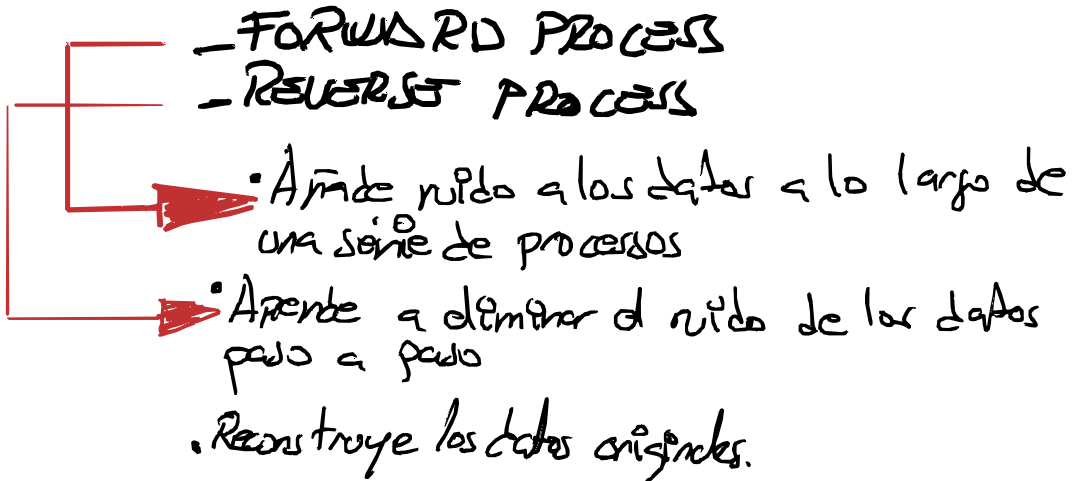
* EMPIEZA CON UN RUIDO ALEATORIO Y A POCO GRADUAMENTE UNA SERIE DE TRANSFORMACIONES

PARA PRODUCIR UNA MUESTRA DE DATOS
COHERENTES.

* EL PROCESO ES SIMILAR A LA SIMULACIÓN DEL
PROCESO FÍSICO DE DIFUSIÓN, EN EL QUE LAS
PARTÍCULAS SE EXTENDEN DESDE REGIONES
DE ALTA CONCENTRACIÓN A REGIONES DE
BAJA CONCENTRACIÓN.

== ¿CÓMO FUNCIONAN? ==

* MEDIANTE:



* ESTE PROCESO DE REMPLAZO DE RUIDO
PUERBO ES LO QUE PERMITE A LOS MODELOS
DE DIFUSIÓN GENERAR MUESTRAS DE
ALTA CALIDAD A PARTIR DE RUIDO ALEATORIO.

== APLICACIONES ==

* GENERACIÓN DE IMÁGENES.

└─▶ • IMÁGENES REALISTAS A PARTIR DE RUIDOS NEUTRALES

* ELIMINA EL RUIDO DE LAS IMÁGENES.

└─▶ • ELIMINA EL RUIDO PARA MEJORAR LA CALIDAD.

* AUMENTO DE DATOS.

└─▶ • GENERACIÓN DE DATOS SINTÉTICOS PARA AUMENTAR LOS CONJUNTOS DE DATOS DE ENTRENAMIENTO.

== DIFFUSION MODEL ==

Define the Diffusion model architecture

```
input_layer = Input(shape=(28, 28, 1))
```

```
x = Conv2D(32, (3, 3), activation='relu',  
          padding='same')(input_layer)
```

```
x = Conv2D(64, (3, 3), activation='relu',  
          padding='same')(x)
```



```
x = Flatten()(x)
```

```
x = Dense(128, activation='relu')(x)
```

```
x = Dense(784, activation='sigmoid')(x)
```

```
output-layer = Reshape((28, 28, 1))(x)
```

```
diffusion_model = Model(input-layer, output-layer)
```

```
# Compile the model
```

```
diffusion_model.compile(optimizer='adam', loss='binary_crossentropy')
```

```
# Summary model
```

```
diffusion_model.summary()
```

```
# Load the data set
```

```
(x_train, _), (x_test, _) = mnist.load_data()
```

```
x_train = x_train.astype('float32') / 255.
```

```
x_test = x_test.astype('float32') / 255.
```

```
x_train = np.expand_dims(x_train, axis=-1)
```

```
x_test = np.expand_dims(x_test, axis=-1)
```

Add noise to images

noise_factor =

```
x_train_noisy = x_train + noise_factor * np.random.  
    normal(loc = 0.0, scale = 1.0,  
    size = x_train.shape)
```

```
x_test_noisy = x_test + noise_factor * np.random.  
    normal(loc = 0.0, scale = 1.0, size =  
    x_test.shape)
```

```
x_train_noisy = np.clip(x_train_noisy, 0., 1.)
```

```
x_test_noisy = np.clip(x_test_noisy, 0., 1.)
```

Train the model

```
diffusion_model.fit(x_train_noisy, x_train,
```

```
    epochs = 50, batch_size = 128,
```

```
    shuffle = True, validation_data = (x_test_noisy,  
    x_test))
```

Predict the denoised images

denoised_images =

diffuser_model.predict(x_test_noisy)

Visualize the results

n = 10 → NUMBER OF DISPLAY
PER ROW

plt.figure(figsize=(20, 6))

for i in range(n):

Display original

ax = plt.subplot(3, n, i + 1)

plt.imshow(x_test[i].reshape(28, 28), cmap='gray')

ax.get_xaxis().set_visible(False)

ax.get_yaxis().set_visible(False)

Display noisy

ax = plt.subplot(3, n, i + 1 + n)

plt.imshow(x_test_noisy[i], reshape(28, 28), cmap='gray')

ax.get_xaxis().set_visible(False)

ax.get_yaxis().set_visible(False)

Display denoised

```
ax = plt.subplot(3, n, i + 1 + 2 * n)
plt.imshow(denoised_image[i].reshape(28, 28),
           cmap='gray')
```

```
ax.get_xaxis().set_visible(False)
ax.get_yaxis().set_visible(False)
```

Unfreeze the top layers of the model

```
for layer in diffusion_model[-4:]:
    layer.trainable = True
```

Compile the model

```
diffusion_model.compile(optimizer='adam', loss=
    'binary_crossentropy')
```

Train the model again

```
diffusion_model.fit(x_train_noisy, x_train,
    epochs = 50, batch_size = 128,
    shuffle = True, validation_data = (x_test_noisy,
    x_test))
```


#####

LABORATORIO VII

MODULO 4

#####

APLICACIÓN DE MODELOS DE DIFUSIÓN

#####

#####

#####

#####

#####

REDES GENERATIVAS

ADVERSARIAS (GAN)

MODULO 4

#####

== INTRODUCCIÓN ==

LAS REDES GENERATIVAS DE CONTRIBUCIÓN GAN, SON UN TIPO REVOLUCIONARIO DE ARQUITECTURA DE REDES NEURONALES DISERDAS PARA GENERAR.

- DATOS SIMETRICOS:

└───▶ QUE SE PASAN MUCHO A LOS DATOS REALES.

- GRAN CAPACIDAD PARA GENERAR IMÁGENES, MÚSICA E INCLUSO TEXTO DE ALTA CALIDAD.

== COMPOSICIÓN ==

* GENERADOR Y DISCRIMINADOR *

• GENERADOR :

- CREA DATOS REALISTAS

• DISCRIMINADOR:

- SU OBJETIVO ES DISTINGUIR ENTRE DATOS REALES Y LOS GENERADOS.

* ESTAS DOS REDES SE ENFRENTAN SIMULTANEAMENTE A TRAVÉS DE UN PROCESO DE ENTRENAMIENTO CONTRAÍDO.

* A TRAVÉS DE ESTE PROCESO CONTRADICTOR, AMBAS REDES MEJORAN CON EL TIEMPO.

* EL RESULTADO DE ESTO ES LA GENERACIÓN DE DATOS MUY REALISTAS.

— ¿CÓMO FUNCIONA? —

● GENERATOR:

- RANDOM NOISE INPUT
- GENERA DATOS SINTÉTICOS
- ESTÁ ENTRENADO PARA ENGAÑAR AL DISCRIMINADOR

● DISCRIMINATOR:

- RECIBE DATOS REALES Y SINTÉTICOS
- INTENTA CLASIFICARLOS DATOS EN REALES O FALSOS
- ESTÁ PREPARADO PARA CLASIFICAR LOS DATOS CON PRECISIÓN.

✗ ESTE CICLO DE ENTRENAMIENTO CONTRADICTORIO CONTINÚA HASTA QUE EL GENERADOR PUEDE PRODUCIR DATOS QUE EL DISCRIMINADOR YA NO DISTINGUE LOS DATOS REALES.

== APLICACIONES ==

* GENERACIÓN DE IMÁGENES REALISTAS
A PARTIR DE RUIDO ALAZAR

* TRADUCCIÓN DE IMAGEN A IMAGEN
CONVIERTES IMÁGENES DE UN DOMINIO A
OTRO :

- CONVIERTES BOCOTOS EN FOTOGRAFÍAS.

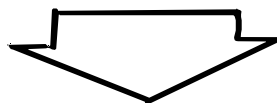
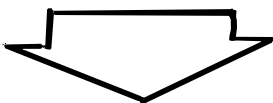
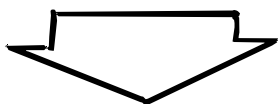
* SÍNTESIS DE TEXTO A IMAGEN:

- GENERA IMÁGENES A PARTIR DE
DESCRIPCIONES TEXTUALES.

* AUMENTO DE DATOS:

- GENERA DATOS SINTÉTICOS PARA
AUMENTAR LOS CONJUNTOS DE DATOS DE
ENTRENAMIENTO.

== CODE EXAMPLE ==



Define the generator model

```
def build_generator():  
    model = Sequential()  
    model.add(Dense(256, input_dim=100))  
    model.add(LeakyRelu(alpha=0.2))  
    model.add(Batch Normalization (momentum=0.9))  
    model.add(Dense(512))  
    model.add(LeakyRelu(alpha=0.2))  
    model.add(Batch Normalization (momentum=0.9))  
    model.add(Dense(1024))  
    model.add(LeakyRelu(alpha=0.2))  
    model.add(Batch Normalization (momentum=0.9))  
    model.add(Dense(28*28*1, activation='tanh'))  
    model.add(Reshape((28, 28, 1)))
```

Define discriminator model

```
def build_discriminator():  
    model = Sequential()  
    model.add(Flatten(input_shape=(28, 28, 1)))
```

```
model.add(Dense(512))  
model.add(LeakyRelu(alpha=0.2))  
model.add(Dense(256))  
model.add(LeakyRelu(alpha=0.2))  
model.add(Dense(1, activation='sigmoid'))  
return model
```

Build and compile discriminator

```
discriminator = build_discriminator()  
discriminator.compile(loss='binary_crossentropy',  
optimizer='adam', metrics=['accuracy'])
```

Build the generator

```
generator = build_generator()
```

Create the GAN by stacking the generator
and the discriminator

```
gan_input = Input(shape=(100,))  
generate_image = generator(gan_input)  
discriminator.trainable = False
```

```
gen_output = discriminator (generate_image)
gen = Model (gen_input, gen_output)
gen.compile(loss = 'binary_crossentropy',
            optimizer = 'adam')
```

* EL DISCRIMINADOR SE
CONVIERTE EN UN MODELO ENTRENABLE
PARA QUE SE ADAPTE EL
GENERADOR DURANTE EL
ENTRENAMIENTO AJUSTADO.

== TRAINING THE GAN ==

load and preprocess the MNIST dataset

```
(x_train, _), (_, _) = mnist.load_data()
```

```
x_train = x_train / 255.0 - 1.
```

```
x_train = np.expand_dims(x_train, axis = -1)
```

Training parameters

```
batch_size = 64
```

```
epochs = 10000
```

```
sample_interval = 1000
```

Adversarial ground truths

real = np.ones((batch-size, 1))

fake = np.zeros((batch-size, 1))

Training loop

for epochs in range(epochs):

idx = np.random.randint(0, x_train.shape[0],
batch-size)

real_images = x_train[idx]

noise = np.random.normal(0, 1, (batch-size,
100))

generate_images = generator.predict(noise)

d_loss_real = discriminator.train-on-batch(
real_images, real)

d_loss_fake = discriminator.train-on-batch(
generate_images, fake)

d_loss = 0.5 * np.add(d_loss_real, d_loss_fake)

Train the generator

noise = np.random(0, 1, (batch-size, 100))

g_loss = gan.train-on-batch(noise, real)

Print the progress

if epochs % sample_interval == 0:

print(f"epoch {D_loss: } d-loss { } { } [

D accuracy: { 100 * d_loss [1] { } [G_loss:
g_loss { }"])

GENERATED
SAMPLES OF
THE NOISE

== EVALUATING GAN ==



def sample_images(epochs):

noise = np.random.normal(0, 1, (25, 100))

gen_images = generator.predict(noise)

gen_images = 0.5 * gen_images + 0.5

fig, axs = plt.subplots(5, 5, figsize=(10, 10))

count = 0

for i in range(5):

for j in range(5):

axs[i, j].imshow(gen_images[count, :, :,
0], cmap = 'gray')

axs[i, j].axis('off')

count += 1

plt.show()

Sample images at the end of training
sample_images (epochs)

* PODEREMOS OBSERVAR COMO LA CALIDAD
DE LAS IMÁGENES GENERADAS MEJORA CON
EL TIEMPO.

#####

TENSORFLOW P2P2

APRENDIZAJE NO SUPERVISADO MODULO 4

#####

== INTRODUCCION ==

- * EL APRENDIZAJE NO SUPERVISADO ES UN TIPO DE MACHINE LEARNING EN EL MODELO QUE SE ENTRENA CON DATOS SIN RESPUESTAS ETIQUETADAS
- * ESTE ENFOQUE ES ESENCIAL PARA DESCUBRIR PATRONES Y ESTRUCTURAS OCULTAS EN LOS DATOS.
- * APLICACIONES:
 - CLUSTERING
 - DIMENSIONALITY REDUCTION
 - ANOMALY DETECTION

== APPLICATIONS ==

* CLUSTERING:

- GRUPO DE PUNTOS SIMILARES

* DIMENSIONALITY REDUCTION:

- REDUCIR LA CANTIDAD DE ENTIDADES EN UN CONJUNTO DE DATOS

* ANOMALY DETECTION:

- LA IDENTIFICACIÓN DE PUNTOS DE DATOS INUSUALES QUE NO SE AJUSTAN AL PATRÓN COMÚN

● SE USAN PARA:

- SEGMENTACIÓN DE CLIENTES
- COMPRESIÓN DE IMÁGENES
- DETECCIÓN DE FRAUDES

== TENSORFLOW TOOLS ==

* NOS OFRECE HERRAMIENTAS PARA IMPLEMENTAR MODELOS DE DEPENDENCIAS NO SUPERVISADO.

* ARCHITECTURAL Y SIMPLAS BIBLIOTECAS

* NOS GESTIONAR VARIAS TAREAS DE
APRENDIZAJE SIN SUPERVISION DE
MANERA EFICIENTE

* NOS CONTRIBUYEN:

- CLUSTERING
- REDUCCION DE LA DIMENSIONALIDAD
(TENSORFLOW)

== CLUSTERING MODEL ==

Load and preprocess the MNIST dataset

```
(x_train, _), (_, _) = mnist.load_data()
```

```
x_train = x_train.astype('float32') / 255.0
```

```
x_train = x_train.reshape(-1, 28*28)
```

Apply K-means clustering

```
Kmeans = KMeans(n_clusters=10)
```

```
Kmeans.fit(x_train)
```

```
labels = Kmeans.labels_
```

Display a few clustered images

```
def display_cluster_images(x_train, labels):
```

```
    plt.figure(figsize=(10, 10))
```

```
    for i in range(10):
```

```
        idx = np.where(labels == i)[0]
```

```
        for j in range(10):
```

```
            plt_idx = i * 10 + j + 1
```

```
            plt.subplot(10, 10, plt_idx)
```

```
            plt.imshow(x_train[idx[j]].
```

```
                        reshape(28, 28), cmap='gray')
```

```
            plt.axis('off')
```

```
plt.show()
```

```
display_cluster_images(x_train, labels)
```

== DIMENSIONALITY
REDUCTION MODEL ==

Define the autoencoder model

```
input_layer = Input(shape=(784,))
```

```
encoded = Dense(64, activation='relu')(input_layer)
```

```
bottleneck = Dense(32, activation='relu')(encoded)
decoded = Dense(64, activation='relu')(bottleneck)
output-layer = (Dense(784, activation='sigmoid'))(
    decoded)
```

```
autoencoder = Model(input-layer, output-layer)
```

Compile the model

```
autoencoder.compile(optimizer='adam', loss='binary_crossentropy')
```

Train the model

```
autoencoder.fit(x_train, x_train, epochs=50,
    batch_size=256, shuffle=True,
    validation_split=0.2)
```

*ESTE PROCESO REDUCE LA
DIFERENCIABILIDAD DE DATOS

Get the compressed representations

```
encoder = Model(input-layer, bottleneck)
compressed-data = encoder.predict(x_train)
```

Use t-SNE for 2D visualization

```
tsne = TSNE(n_components=2)
```

```
compressed_2d = tsne.fit_transform(compressed_data)
```

Plot the compressed data

```
plt.scatter(compressed_2d[:, 0], compressed_2d[:, 1], c=labels, cmap='viridis')
```

```
plt.colorbar()
```

```
plt.show()
```


#####

LABORATÓRIO IX

MODULO 4

#####

DESENVOLVO DE GAN CON VERAS

#####

#####

#####

#####

Resumen y aspectos destacados:

Aprendizaje no supervisado y Modelos generativos en Keras

Enhorabuena Ha completado este módulo. A estas alturas del curso, ya lo sabe:

- El Aprendizaje no supervisado es un tipo de aprendizaje automático en el que un algoritmo encuentra patrones en los datos sin etiquetas ni resultados predefinidos.
- El Aprendizaje no supervisado puede clasificarse a grandes rasgos en dos tipos: agrupamiento y reducción de dimensionalidad.
- Los autocodificadores constan de dos partes principales: codificador y decodificador.
- Las redes generativas adversarias (GAN) constan de dos redes, la generadora y la discriminadora, que compiten entre sí en un juego de suma cero.
- La red generadora genera nuevas instancias de datos que se asemejan a los datos de entrenamiento.
- La red discriminadora evalúa la autenticidad de los datos generados.
- Los autocodificadores son herramientas versátiles para diversas tareas, como la eliminación de ruido de los datos, la reducción de dimensionalidad y el aprendizaje de características.
- La arquitectura básica de un autocodificador incluye tres componentes principales: codificador, cuello de botella y decodificador.
- Existen distintos tipos de autocodificadores: autocodificadores básicos, autocodificadores variacionales (VAE) y autocodificadores convolucionales.
- Los modelos de difusión son potentes herramientas para tareas generativas, capaces de producir muestras de datos de alta calidad y de mejorar la calidad de las imágenes.
- Son modelos probabilísticos que generan datos refinando iterativamente una muestra inicial ruidosa.
- El proceso es similar a la simulación del proceso físico de difusión, en el que las partículas se dispersan desde regiones de alta concentración a regiones de baja concentración.
- Los modelos de difusión funcionan definiendo un proceso de avance y otro de retroceso.
- Las GAN son un tipo revolucionario de arquitectura de redes neuronales diseñadas para generar datos sintéticos que se asemejan mucho a los datos reales.
- Las GAN constan de dos componentes principales: un generador y un discriminador.
- Estas dos redes se entrenan simultáneamente mediante un proceso de entrenamiento adversario.
- Este bucle de entrenamiento adversario continúa hasta que el generador produce datos que el discriminador ya no puede distinguir de los datos reales.
- El Aprendizaje no supervisado es un potente enfoque para descubrir patrones ocultos en los datos, y TensorFlow proporciona herramientas robustas para facilitar estas tareas.

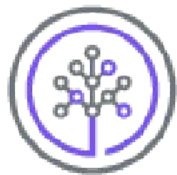
- Las aplicaciones más comunes incluyen clustering, reducción de dimensionalidad y detección de anomalías.
- Estas aplicaciones se utilizan ampliamente en diversos ámbitos, como la segmentación de clientes, la compresión de imágenes y la detección de fraudes.

Module 4 Glossary: Unsupervised Learning and Generative Models in Keras

Welcome! This alphabetized glossary contains many terms used in this course. Understanding these terms is essential when working in the industry, participating in user groups, and participating in other certificate programs.

Term	Definition
Anomaly detection	A type of unsupervised learning used to identify unusual data points that do not fit the general pattern in a dataset.
Adam Optimizer	An optimization algorithm that can be used instead of the classical stochastic gradient descent procedure to update network weights iteratively based on training data.
Adversarial training	A training method in GANs where the generator and discriminator networks are trained simultaneously, with the generator aiming to fool the discriminator, and the discriminator trying to accurately classify real and fake data.
Autoencoders	A type of neural network used to learn efficient representations of data, often for dimensionality reduction or feature learning.
Binary crossentropy loss	A loss function used in binary classification tasks, often utilized in training neural networks to measure the difference between predicted and actual outputs.
Bottleneck	The central, most compressed layer in an autoencoder that contains the most critical features of the input data.
Clustering	A method in unsupervised learning that involves grouping data points into clusters, where data points in the same cluster are more similar to each other.
Convolutional autoencoders	A type of autoencoder that uses convolutional layers, making it particularly effective for tasks involving image data.
Convolutional neural network (CNN)	A type of deep neural network commonly used in image processing tasks, known for its ability to capture spatial hierarchies in images.
Data augmentation	A technique used to increase the diversity of data available for training models by generating new synthetic data, often by applying transformations to existing data.
Decoder	The part of an autoencoder that reconstructs the input data from the compressed latent space representation.
Denoising	The process of removing noise from data, such as images, to improve their quality.
Diffusion	A physical process where particles spread from regions of high concentration to low concentration; in diffusion models, this concept is simulated to generate or enhance data.
Diffusion model	A type of probabilistic generative model that iteratively refines noisy data to produce high-quality samples, often used in image generation.
Dimensionality reduction	A process in unsupervised learning that reduces the number of random variables under consideration by obtaining a set of principal variables.
Discriminator network	In a generative adversarial network (GAN), this network evaluates the authenticity of the generated data, distinguishing between real and fake data.
Encoder	The part of an autoencoder that compresses the input data into a latent-space representation.
Epochs	In machine learning, an epoch refers to one complete pass of the training dataset through the learning algorithm.
Feature learning	A set of techniques that allow a machine to automatically discover the representations needed for feature detection or classification from raw data.
Forward process	In diffusion models, the process of gradually adding noise to data over a series of steps.
Functional API	A way to build neural networks in Keras that allows for more flexible model architectures than the Sequential API.
Generative adversarial networks (GANs)	A class of neural networks where two networks, the generator and the discriminator, compete against each other, leading to the generation of realistic data.
Generator	In GANs, the neural network creates synthetic data from random noise, aiming to produce data that closely resembles real data.
Image-to-image translation	A task in computer vision where an image from one domain is transformed into an image in another domain, such as converting a sketch into a photo.
K-means algorithm	A popular clustering technique that partitions a dataset into distinct groups based on the features of the data points.
Keras	An open-source software library that provides a Python interface for artificial neural networks and is used to create deep learning models.
Latent-space representation	The compressed version of input data generated by the encoder in an autoencoder.
MNIST dataset	A large database of handwritten digits that is commonly used for training image processing systems and machine learning models.
Modified National Institute of Standards and Technology (MNIST) Dataset	A large database of handwritten digits commonly used for training various image processing systems.
Neural network architecture	The structured layout of a neural network, including its layers, connections, and the flow of information within it.

Term	Definition
Normalization	The process of scaling input features so they have a mean of zero and a standard deviation of one, often used to improve the performance of neural networks.
Principal component analysis (PCA)	A dimensionality reduction technique that transforms data into a set of linearly uncorrelated variables called principal components.
Probabilistic model	A model that incorporates randomness and uncertainty, often used to predict distributions or simulate processes that have inherent variability.
Reverse process	In diffusion models, the process of removing noise step by step to reconstruct the original data from a noisy sample.
Stochastic gradient descent (SGD)	An optimization method that adjusts weights iteratively based on a subset of training data, used in training neural networks.
Supervised learning	A type of machine learning where the algorithm is trained on labeled data, meaning the outcome or target variable is known during training.
t-distributed stochastic neighbor embedding (t-SNE)	A dimensionality reduction technique used for visualizing high-dimensional data by giving each datapoint a location in a two or three-dimensional map.
TensorFlow	An open-source machine learning library developed by Google, widely used for building and training machine learning models.
Text-to-image synthesis	A process where a model generates an image based on a textual description provided as input.
Training epoch	A single pass through the entire training dataset during the training process of a machine learning model.
Unsupervised learning	A type of machine learning that finds patterns in data without any labels or predefined outcomes.
Variational autoencoders (VAEs)	A type of autoencoder that introduces probabilistic elements to generate new data samples, often used in generative models.
Zero-Sum Game	A situation in competitive contexts where gain or loss of participants is exactly balanced by the losses or gains of another participant.



Skills Network

#####

TÉCNICAS AVANÇADAS DE KERAS

MÓDULO 5

#####

== INTRODUÇÃO ==

* KERAS OFERECE TÉCNICAS QUE PODEM
ACELERAR SIGNIFICATIVAMENTE O
PROCESSO DE DESENVOLVIMENTO DE MODELOS

== TÉCNICAS ==

- * BUCLES DE ENTREALIMENTAÇÃO COSTOMIZADA
- * CAPAS ESPECIALIZADAS
- * FUNÇÕES AVANÇADAS DE DESENVOLVIMENTO DE CAMADA

== CUSTOM TRAINING LOOPS ==

* ESTOS LOOPS BRINDAN MÁS CONTROL SOBRE EL PROCESO DE ENTRENAMIENTO.

- PERMITE ADAPTAR EL PROCESO DE ENTRENAMIENTO A SUS NECESIDADES.

- ÚTIL PARA IMPLEMENTAR ESTRATEGIAS DE ENTRENAMIENTO COMPLEJAS O FUNCIONES DE PÉRDIDA PERSONALIZADAS.

== CUSTOM TRAINING LOOPS ==

CODE

Create a simple model

```
model = Sequential([Dense(64, activation='relu'),  
                    Dense(10)])
```

Custom training loop

```
optimizer = tf.keras.optimizers.Adam()
```

```
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

load or create your training dataset here

Example

```
x_train = tf.random.uniform((100, 10))
```

```
y_train = tf.random.uniform((100,)) maxval = 10,  
dtype = tf.int64)
```

```
train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train))  
batch(32)
```

```
for epoch in range(10):
```

```
    with tf.GradientTape() as tape:
```

```
        logits = model(x_batch, training = )
```

```
        loss = loss_fn(y_batch, logits)
```

```
        grads = tape.gradient(loss, model.  
                                trainable_weights)
```

```
        optimizer.apply_gradients(zip(grads,  
                                       model.trainable_weights))
```

```
print(f'Epoch {epoch + 1}, Loss: {loss.numpy()  
      }')
```

== SPECIALIZED LAYERS ==

* CAPAS ESPECIALIZADAS PARA AMPLIAR LA FUNCIONALIDAD DE UN MODELO:

- CUSTOM DENSE LAYERS

- LAMBDA LAYERS

* ESTAS CAPAS PUEDEN IMPLEMENTARSE:

- FUNCIONES DE ACTIVACIÓN
ADAPTADAS PERSONALIZADAS

QUE NO ESTÉN DISPONIBLES EN
LA BIBLIOTECA ESTÁNDAR Keras

== CUSTOM LAYER ==

```
class CustomDenseLayer(Layer):
```

```
    def __init__(self, units=32):
```

```
        super(CustomDenseLayer, self).__init__()
```

```
def build (self, input_shape):  
    self.W = self.add_weight (shape = (input_shape  
        [-1], self.units),  
        initializer = 'random-normal',  
        trainable = True)
```

```
self.b = self.add_weight (shape = (self.units, 1),  
        initializer = 'zeros',  
        trainable = True)
```

```
def call (self, inputs):  
    return tf.matmul(inputs, self.W) + self.b
```

Usage in a model

```
model = Sequential ([Custom Dense Layer (64),  
                    Dense (10)])
```

== CALLBACK FUNCTIONS ==

* FUNCIONES DE DEVOLUCIÓN DE LLAMADAS

- PARA PERSONALIZAR Y AMPLIAR EL COMPORTAMIENTO DE SU PROCESO DE ENTRENAMIENTO DE MODELOS

* PERMISOS

- SUPERVISAR EL ENTRENAMIENTO (capacitación)
- IMPLEMENTAR PARADAS AUTOMÁTICAS
- GUARDAR PUNTOS DE CONTROL (checkpoint)
- DEVOLUCIONES DE LLAMADAS PERSONALIZADAS

== CALLBACK FUNCTIONS ==

CODE

```
class Custom Callback (Callback):  
    def on_epoch_end (self, epoch, logs=None):  
        logs = logs or {}
```



```
print (f'End of epoch {epoch}, loss: {loss.  
get("loss")} {accuracy: } logr.set(  
"accuracy") {') )
```

Usage in model training

```
model.compile(optimizer='adam', loss=  
'sparse_categorical_crossentropy',  
metrics=['accuracy'])
```

```
model.fit(train_dataset, epochs=10,  
callbacks=[CustomCallback()])
```

== MODEL OPTIMIZATION ==

* LA OPTIMIZACIÓN DE LOS MODELOS

ES CRUCIAL PARA MEJORAR EL RENDIMIENTO
DE LOS MODELOS

== HERRAMIENTAS ==

* HERRAMIENTAS DE OPTIMIZACIÓN:

- ENTRENAMIENTO DE PRECISIÓN MIXTA
(Mixed Precision Training)
- KIT DE HERRAMIENTAS DE OPTIMIZACIÓN DE TENSORFLOW.

== Mixed Precision Training ==

Enable mixed precision

```
mixed_precision.set_global_policy('mixed_float16')
```

Model definition

```
model = Sequential([Dense(64, activation='relu'),  
                    Dense(10)])
```

```
model.compile(optimizer='adam', loss=  
              'sparse_categorical_crossentropy',  
              metrics=['accuracy'])
```

Training the model

```
model.fit(train_dataset, epochs=10)
```


#####

CUSTOM TRAINING LOOPS

NOTES

MODULO 5

#####

== ESTRUTURA BÁSICA ==

- * CONJUNTO DE DADOS (data set)
- * MODELO (Model)
- * OPTIMIZADOR (optimizer)
- * FUNÇÃO DE PERDA (loss)

== CUSTOM TRAINING LOOP ==

① PREPARE A SIMPLE DATASET

prepare a simple dataset

$(x_train, y_train), (x_test, y_test) = tf.keras.datasets.mnist.load_data()$

$x_train, x_test = x_train / 255.0, x_test / 255.0$

$train_dataset = tf.data.Dataset.from_tensor_slices((x_train, y_train)).batch(32)$

2. DEFINING THE LOSS FUNCTION AND OPTIMIZER

DEFINE LOSS FUNCTION & OPTIMIZER

```
loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)
```

```
optimizer = tf.keras.optimizers.Adam()
```

→ indica que tan bien las predicciones del modelo coinciden con los etíquetos verdaderos.

Optimiza los pesos del modelo para minimizar la pérdida

== IMPLEMENTING ==

Create a simple model with a Flatten layer

```
model = Sequential([
```

```
    Flatten(input_shape=(28,28))
```

```
    Dense(128, activation='relu'),
```

```
    Dense(10)
```

```
])
```

Define the loss function and optimizer

loss_fn = tf.keras.losses.SparseCategoricalCrossentropy(from_logits = True)

optimizer = tf.keras.optimizers.Adam()

Custom training loop

epochs = 5

for epoch in range(epochs)

print(f'Start of epoch {epoch + 1}')

for step, (x_batch_train, y_batch_train) in enumerate(train_dataset):

with tf.GradientTape() as tape:

logits = model(x_batch_train, training=True)

loss_value = loss_fn(y_batch_train, logits)

gradients = tape.gradient(loss_value, model.trainable_weights)

optimizer.apply_gradients(zip(gradients, model.trainable_weights))

trainable_weights))

if step % 200 == 0:

print(f'Epoch {epoch + 1} Step {step} :
Loss = {loss_value.numpy()}')

INITIALIZE:

1. Gradient Tape : Es una API de TensorFlow

Se puede ver el avance para a través de la red,
lo que permite a TensorFlow registrar todas las
operaciones en las variables observadas.

== BENEFICIOS ==

* CONTROL MÁS DETALLADO SOBRE EL PROGRESO
DE ENTRENAMIENTO.

* CUSTOMIZAR :

- FUNCIONES DE PÉRDIDA
- OPTIMIZACIÓN DE PÉRDIDA

* REGISTRO Y MONITOREO DIVERSOS

* IMPORTANTES PARA INVESTIGACIÓN y
DESARROLLO.

* INTEGRACIÓN CON GADPS Y OPERACIONES
DERIVADAS.

#####

LABORATORIO I

MODULO 5

#####

BUCLAS DE ENTRENAMIENTO

CON
RERDS

#####

#####

#####

#####

AJUSTES DE HIPERPARÁMETROS

KERAS TUNER

MODULO 5

#####

== INTRODUCCIÓN ==

HYPERPARAMETER TUNING

ES UN PASO CLAVIC EN EL PROCESO DE ML.

X AYUDA A OPTIMIZAR EL MODELO AL ENCONTRAR
EL MEJOR CONJUNTO DE HIPERPARÁMETROS

X KERAS TUNER → SIMPLIFICA ESTE PROCESO AL
PROPORCIONAR UNA INTERFAZ FÁCIL DE USAR.

== ¿QUÉ SON? ==

X LOS HIPERPARÁMETROS SON LAS VARIABLES
QUE REGEN EL PROCESO DE ENTRENAMIENTO
DE UN MODELO.

X A DIFERENCIA DE LOS PARÁMETROS DEL MODELO QUE SE APRENDEN DURANTE EL ENTRENAMIENTO. LOS EJEMPLOS INCLUYEN:

- LÍMITES DE APRENDIZAJE
- EL TAMAÑO DEL COJO
- CANTIDAD DE CAPAS O NÚMERO DE UNA RED NEURONAL.

== KERAS TUNING ==

- ALGORITMOS DE BÚSQUEDA:

- BÚSQUEDA ALGORITMO
- HIPERBANDA
- OPTIMIZACIÓN BAYESIANA

- ESTOS ALGORITMOS BUSCAN DE MANERA EFICIENTE LOS MEJORES VALORES DE HIPERPARÁMETROS.
PARA SU MODELO

== CODE KERAS TUNER ==

INSTAL LIBRARIES

```
import keras-tuner as kt  
from tensorflow.keras.models import Sequential  
from tensorflow.keras.layers import Dense, Flatten  
from tensorflow.keras.datasets import mnist  
from tensorflow.keras.optimizers import Adam
```

Define function to build model hyperparameters.

```
def build_model(hp):
```

```
    model = Sequential([
```

```
        Flatten(input_shape=(28, 28)),
```

```
        Dense(units=hp.Int('units', min_value=32,  
max_value=512, step=32), activation='relu'),
```

```
        Dense( , activation='softmax')
```

```
    ])
```

model.compile (

optimizer=Adam (learning_rate.Float (
 'learning_rate', min-value = $1e-4$, max-value = $1e-2$, sampling = 'Log')),

loss='sparse_categorical_crossentropy' ,

metrics = ['accuracy']

)

return model

== CONFIGURING SEARCH ==

knr = Kt.RandomSearch (

build_model,

objective = 'val_accuracy',

max_trials = 10,

executions_per_trial = 2,

directory = 'my_dir',

project_name = 'Intro-to-Kt'

)

EL SPREADSHEET
EXPLAINS
EXPLORE THE
HPY ENCAPSULATION
AND METRICS
CONFIGURATION

== RUNNING THE HYPERPARAMETER SEARCH ==

```
(x_train, y_train), (x_val, y_val) = mnist.load_data()
x_train, x_val = x_train / 255.0, x_val / 255.0
tuner.search(x_train, y_train, epochs = 5,
             validation_data = (x_val, y_val))
```

== ANALYZING THE RESULTS ==

```
best_hp =
tuner.get_best_hyperparameters(num_trials = 1)[0]

print(f"""
The optimal number of units in the first dense layer
is {best_hps.get('units')}.
The optimal learning rate for the optimizer is
{best_hps.get('learning_rate')}.
""")
```

```
model = tuner.hypermodel.build(best_hps)
model.summary()
```

== TRAINING THE OPTIMIZED MODEL ==

```
model.fit(x_train, y_train, epochs=10,
          validation_split=0.2)
```

```
test_loss, test_acc = model.evaluate(x_test,
                                       y_test)
```

```
print(f'Test accuracy: {test_acc}')
```


#####

LABORATÓRIO ^{VI}

MODULO 5

#####

AJUSTES DE HIPERTENSÍMETROS

UU

HERAS

#####

#####

#####

#####

#####

OPTIMIZACIÓN DE MODELOS KERAS

MODULO 5

#####

== INTRODUCCIÓN ==

LA OPTIMIZACIÓN DE MODELOS TIENE
COMO OBJETIVO:

- MEJORAR EL RENDIMIENTO
- MEJORAR LA EFICIENCIA DE LAS REDES
- MEJORAR LA ESCALABILIDAD

LOS MODELOS OPTIMIZADOS:

- TIEMPOS DE ENTRENO MAS RÁPIDOS
- MEJOR USO DEL HARDWARE
- MAYOR PRECISIÓN

== TÉCNICAS ==

- * INICIALIZACIÓN DEL PESO
(Weight Initialization)
- * PROGRAMACIÓN DE TASAS DE APRENDIZAJE
(Learning rate scheduling)
- * NORMALIZACIÓN DE LOTES
(Batch Normalization)

== WEIGHT INITIALIZATION ==

- * EFECTOS A LA CONVERGENCIA Y EL RENDIMIENTO DE UNA RED
- * MÉTODOS DE INICIALIZACIÓN:
 - XAUPTER (GLOPOT) → INITIALIZATION
 - He → " 

* CON ESTOS MÉTODOS SURTEN PROBLEMAS

(OJO:

- VANISHING
- EXPLODING GRADIENTS.

== EJEMPLOS 'He' ==

Define the model with 'He' initialization

```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(128, activation='relu',  
          kernel_initializer=HeNormal()),  
    Dense(10, activation='softmax')  
])
```

== LEARNING RATE SCHEDULING ==

Load the MNIST dataset

```
(x_train, y_train), (x_val, y_val) = mnist.load_data()
```

Normalize the input data

```
x_train = x_train.astype('float32') / 255.0  
x_val = x_val.astype('float32') / 255.0
```

Reshape the input data (if necessary)

```
x_train = x_train.reshape(-1, 28, 28)  
x_val = x_val.reshape(-1, 28, 28)
```

Define learning rate scheduler

```
def scheduler(epoch, lr):
```

```
    if epoch < 10:  
        return lr
```

```
    else:
```

```
        return float(lr * 1.1 ** math.exp(-0.1))
```

```
lr_scheduler = LearningRateScheduler(scheduler)
```

MODEL TRAINING

Compile the model

```
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
```

Train the model with Learning Rate scheduling

```
history = model.fit(x_train, y_train,  
                    validation_data=(x_val, y_val),  
                    epochs=20,  
                    callbacks=[lr_scheduler])
```

Evaluate the model

```
model.compile(optimizer=Adam(), loss='categorical_crossentropy', metrics=['accuracy'])
```

```
history = model.fit(x_train, y_train,  
                    validation_data=(x_val, y_val),  
                    epochs=20)
```

== BATCH NORMALIZATION ==

* SE UTILIZA PARA NORMALIZAR LA CAPA DE ENTRADA AJUSTANDO Y ESCALANDO LAS ACTIVACIONES.

Define a simple model with Batch Normalization.

```
model = Sequential([  
    Flatten(input_shape=(28, 28)),  
    Dense(128, activation='relu'),  
    BatchNormalization(),  
    Dense(10, activation='softmax'),  
])
```

== MIXED PRECISION TRAINING ==

* IMPLICA EL USO DE TIPOS DE PUNTO FLUJANTE DE 16 y 32 bits PARA AUMENTAR EL RENDIMIENTO EN LAS GPU MODERNAS.

Enable mixed precision

```
policy = mixed_precision.Policy('mixed_float16')  
mixed_precision.set_global_policy(policy)
```

== MODEL PRUNING ==

LA RUTA DEL MODELO PRUNING ES LA CANTIDAD DE PARÁMETROS DE UN MODELO AL ELIMINAR LAS CONEXIONES O NEURONAS MENOS IMPORTANTES.

```
prune_low_magnitude = tfmot.sparsity.keras.  
prune_low_magnitude
```

Apply pruning to the model

```
pruning_params = {'pruning_schedule':
```

```
tfmot.sparsity.keras.PolynomialDecay (initial_sparsity=  
0.0,
```

```
final_sparsity=0,
```

```
begin_step=200) {
```

```
model_pruned = prune_low_magnitude(model, xxpruning)
```

Quantization

* REDUCE LA PRECISIÓN DE LOS NÚMEROS UTILIZADOS PARA REPRESENTAR LOS PESOS DE LOS MODELOS.

* AYUDA A IMPLEMENTAR MODELOS EN DISPOSITIVOS PERIFÉRICOS AL REDUCIR EL USO DE MEMORIA Y EL TIEMPO DE EJECUCIÓN

Example Code for Quantization

```
converter = tf.lite.TFLiteConverter.from_keras_model(model)
converter.optimizations = [tf.lite.Optimize.DEFAULT]
quantized_model = converter.convert()
```

Save the quantized model

```
with open('quantized_model.tflite', 'wb') as f:
    f.write(quantized_model)
```


#####

OPTIMIZACIÓN DE MODELOS

CON TENSORFLOW MODULO 5

#####

INTRODUCCIÓN

- ENTRENAMIENTO DE PRECISIÓN MIXTA
(Mixed Precision Training)
- DESTILACIÓN DE CONOCIMIENTOS
(Knowledge distillation)
- TÉCNICAS AVANZADAS DE OPTIMIZACIÓN
POST-ENTRENO
- MODEL PRUNING - (poda)

== KINET PRECISION TRAINING ==

- * ΔPROUCHO LOS TIPOS DE PUNTO FLUJANTE DE 16 y 32 bits
- * ACELERO EL ENTRENAMIENTO
- * REDUCE EL USO DE MEMORIA
- * REDUCE EL ANCHO DE BANDA DE LA MEMORIA
- * PRECISIÓN INTERIOR SIEMPRE QUE ES POSIBLE
- * LOS MODELOS SE ENTRENAN MÁS RÁPIDAMENTE
- * SIN PÉRDIDA DE PRECISIÓN

== KNOWLEDGE DISTILLATION ==

- * ENTRENAR A UN MODELO "ESTUDIANTE" MÁS PEQUEÑO PARA IMITAR EL COMPORTAMIENTO DE UN MODELO "TEACHER" MÁS GRANDE

* EL MODELO "TEACHER":

- RED NEURONAL PROFUNDA Y COMPLEJA

* MODELO "STUDENT":

- RED PEQUEÑA Y SENCILLA QUE APRENDE DE "TEACHER"

✓ ESTA TÉCNICA REDUCE EL TAMAÑO DEL MODELO

✓ TIEMPO DE INFERENCIA MÁS RÁPIDO

✓ RECUPERAS EL RENDIMIENTO DE UN MODELO MÁS COMPLEJO.

== KNOWLEDGE DISTILLATION ==

Teacher model (simple model)

```
teacher_model = models.Sequential([  
    layers.Input(shape=(28, 28)),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(10, activation='softmax'),  
])
```

```
teacher_model.compile(optimizer=optimizer, Adam(),  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy'])
```

Student model (Simpler model)

```
student_model = model.Sequential([  
    layers.Input(shape=(28, 28)),  
    layers.Flatten(),  
    layers.Dense(128, activation='relu'),  
    layers.Dense(10, activation='softmax'),  
])
```

```
teacher_model.compile(optimizer=optimizer, Adam(),  
    loss='sparse_categorical_crossentropy',  
    metrics=['accuracy'])
```

Distillation loss teacher to student

```
def distillation_loss(teacher_logits, student_logits,  
    temperature = 3):
```

```
    teacher_probs = tf.nn.softmax(teacher_logits / temperature)  
    student_probs = tf.nn.softmax(student_logits / temperature)  
    return
```

```
tf.nn.reduce_mean(tf.keras.losses.categorical_crossentropy (teacher_probs, student_probs))
```

Train student

```
def train_student (student, teacher, x_train, y_train, batch_size = 32, epochs = 2, temperature = 3):
```

```
    for epoch in range (epochs):
```

```
        num_batches = len(x_train) // batch_size
```

```
        for batch in range (num_batches):
```

```
            x_batch = x_train [batch * batch_size : (batch + 1) * batch_size]
```

```
            ... ..
```

TERMINO DI VOR EN EL
LABORATORIO



Module 5 Glossary: Advanced Keras Techniques and Course Project

Welcome! This alphabetized glossary contains many terms used in this course. Understanding these terms is essential when working in the industry, participating in user groups, and participating in other certificate programs.

Term	Definition
Batch normalization	A technique that normalizes the inputs of each layer to improve the training stability and speed.
Callback functions	A powerful tool to customize and extend the behavior of your model training process.
Custom dense layer	A standard fully connected neural network layer.
Custom training loops	Custom training loops are user-defined procedures in machine learning that give developers more control over the training process. They allow the developer to tailor the training process to specific needs.
Epoch	Refers to one complete pass through the entire training dataset.
fit method	A high-level function commonly used in Keras to train a model.
GradientTape	A feature in TensorFlow that provides an easy way to record operations for automatic differentiation.
hp.Float function	A function used for floating-point hyperparameters.
hp.Int function	Function used for integer hyperparameters
Hyperparameter tuning	A crucial step in the machine learning pipeline that helps optimize model performance by finding the best set of hyperparameters.
Hyperparameters	Variables that govern the training process of a model.
Keras Tuner	A library that helps automate the process of hyperparameter tuning. It provides several search algorithms, including Random Search, Hyperband, and Bayesian Optimization.
Lambda layer	Used for creating custom operations that are simple enough to be defined using a lambda function or a small custom function.
Loss function	Measures how well the predictions made by the model match the true labels.
Mixed precision training	An optimization tool that leverages both 16-bit and 32-bit floating-point types to accelerate model training and reduce memory usage.
MNIST database	A large database of handwritten digits that is commonly used for training various image processing systems.
Model optimization	Refers to a set of techniques and processes that aim to improve the performance and efficiency of your neural networks.
Model pruning	An optimization tool involves removing less important weights from the model, reducing the model size, and improving inference speed.
Optimizer	Updates the weights of the model to minimize the loss.
prune_low_magnitude	A function that prunes the weights based on their magnitude.
Quantization	A technique that converts model weights and activations from floating-point to lower bit-width representations, such as 8-bit integers.
ReLU	An activation function that outputs the input directly if positive; otherwise, it outputs zero. Commonly used in hidden layers.
TensorFlow Model Optimization Toolkit	Provides easy-to-use APIs for pruning during training.
Xavier (Glorot) and He initialization methods	Methods that set the initial weights in a way that avoids issues like vanishing or exploding gradients.



Skills Network

#####

APRENDIZAJE POR (RL)

REFUERZO

MODULO 6

#####

== INTRODUCCIÓN ==

* LOS AGENTES INTERACTUEN CON UN ENTORNO:

AGENTS

ENVIRONMENT

- LOS AGENTES EN ÚLTIMA INSTANCIA TOMAN LA DECISIÓN

Ejem PLO:

- ¿DÓNDE PONER LOS ANUNCIOS EN UNA WEB?

• EL AGENTE SERÍA EL PROGRAMA QUE TOMA LA DECISIÓN SOBRE DÓNDE COLOCAR EL ANUNCIO.

• EL ENTORNO ES EL MUNDO POR EL QUE SE MUEVE EL AGENTE:

- ADEPTES : TÁCTICO → ENVIRONMENT
JUGADOR → AGENT.

* LAS ACCIONES QUE TOMEMOS TENDRÁN UN IMPACTO EN EL AMBIENTE, LO QUE A SU VEZ AFECTARÁ A LOS AGENTES A TRAVÉS DE RECOMPENSAS.

- UN AGENTE HACE UN MOVIMIENTO EN LA PARTIDA DE AJEDREZ, MUEVE ALGUNO DEL ENTORNO DE NUESTRO TABLERO.
- SI ESA JUGADA NOS PERMITIÓ CONSEGUIR MÁS PUNTOS O GANAR LA PARTIDA, ESTE SERÍA UN EJEMPLO DE RECOMPENSA.

(Rewards)

• NUESTRO SISTEMA APRENDERÍA QUE LAS MEDIDAS TOMADAS FUERON BUENAS.

* RECOMPENSAS:

- SON DESCUOCIDAS.
- EL AGENTE DEBE CALCULARLAS
- SE NECESITAN MUCHOS PASOS PARA LLEGAR A ESA FASE DE RECOMPENSAS.

* ESTOS PROCESOS SE REPITEN DE FORMA DINÁMICA.

- DE ESTE MODO LAS MÁQUINAS APRENDEN CONTINUAMENTE COMO ESTÁNDAR LAS PERSONAS CON EL TIEMPO.

- * LOS JUEGOS EN DEEP LEARNING SE DEBEN A LOS JUEGOS EN (RL) & APRENDIZAJE POR REFORZO

- * EN 2013, LOS DESARROLLADORES DE DEEP HAD DESARROLLADO EL SISTEMA PARA JUGAR A LOS JUEGOS DE ATARI² Y DERROTAR A LOS HUMANOS EN LOS JUEGOS

- * EN 2017, EL SISTEMA AlphaGo DERROTÓ AL CAMPEÓN DEL MUNDO EN GO.

- POR PRIMERA VEZ LAS MÁQUINAS SE COMPARARON EN CAMPEONATOS, EN CAMPEONATOS HUMANOS

- UN JUEGO COMPLETO COMO EL GO MEDIANTE (RL).

== OVERVIEW ==

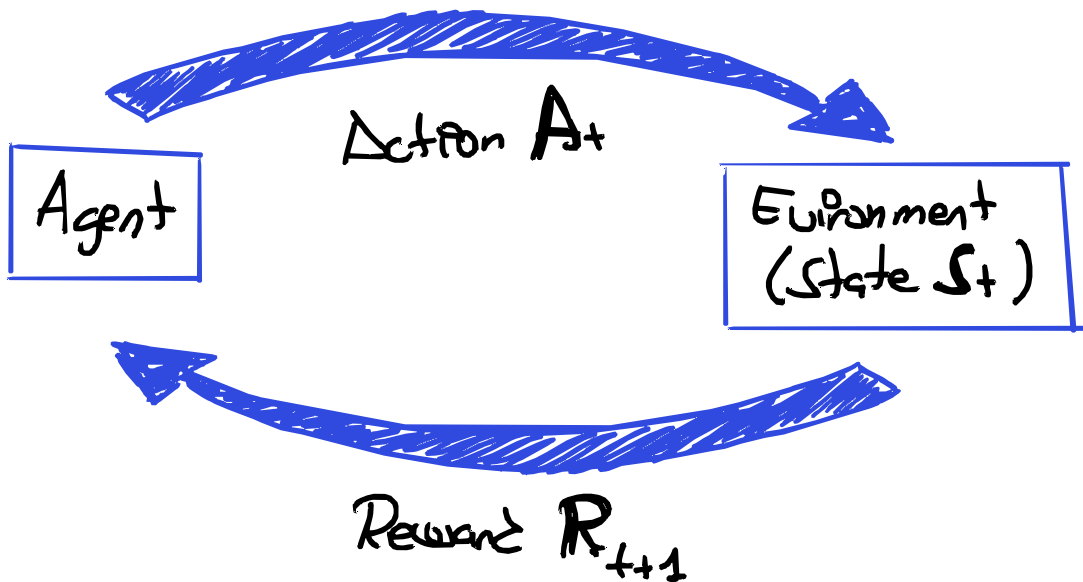
* LOS ALGORITMOS Reinforcement Learning (RL) HAN SIDO COMPROBADOS DEBIDO A LAS IMPORTANTES REQUISITOS COMPUTACIONALES Y DE DATOS.

* HAY INFINIDAD DE POSIBLES COMBINACIONES EN CADA MOMENTO, EN CADA PERSONA QUE VISITA UNA WEB, ETC...

* ES SISTEMA DE RECOMPENSAS PARA TODA ESA CANTIDAD DE INFORMACIÓN PARA ENTRENAR EL MODELO.

== SISTEMA ==

- LA IDEA ES QUE LOS AGENTES TOMEN UNA DECISIÓN
- ESA ACCIÓN AFECTA AL ENTORNO ACTUAL
- LOS CAMBIOS DE ESTE ENTORNO SE TRANSMITEN A LOS AGENTES EN FORMA DE RECOMPENSAS.



* SI EL RESULTADO ES POSITIVO EN RELACIÓN CON NUESTRO SISTEMA DE RECOMPENSAS, LAS ACCIONES DEL AGENTE SE REFUERZAN.

* Y UPONERCA PARA RECOMPENSAS NEGATIVAS EL AGENTE NO TOMARÁ LAS MISMAS MEDIDAS

== SOLUCIÓN E IMPLEMENTACIÓN ==

* LAS PROBLEMAS DE RL VARÍAN SIGNIFICATIVAMENTE.

— LAS SOLUCIONES REPRESENTAN UNO
POLICY MEDIANTE LA CUAL LOS
AGENTES ELABORAN ACCIONES EN RESPUESTA
AL ESTADO ACTUAL (STATE).

— LOS AGENTS SUELEN TRABAJAR PARA
MAXIMIZAR

× ESTO ES LA DIFERENCIA DE LOS
PROBLEMAS TÍPICOS DE ML PORQUE,
A DIFERENCIA DE LO QUE CORRESPONDE
LAS LABELS, LAS REWARDS NO
SE CONVERGEN, Y A MENUDO SON MUY
RUIDOSAS

× ES POSIBLE QUE NO ESPANTAS EN TODO
MOMENTO SI LAS ACCIONES GENERAN
RECOMPENSAS INMEDIATAS O, INCLUSO NO
LO HACERÁN SI LAS RECOMPENSAS AUTENTICAMENTE
CONDUCEAN A LOS OBJETIVOS MÁS AMPLIOS
DE NUESTRA RED.

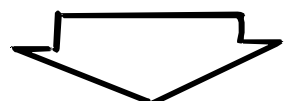
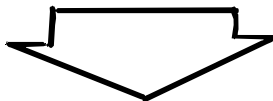
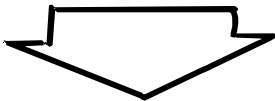
* EN LOS PROBLEMAS TÍPICOS DE ML,
LAS SOLUCIONES PERMANECEN ESTÁTICAS,
CON (RL) A MENUDA QUE LAS ACCIONES
DEPENDAN DEL ENTORNO, EL ESTADO CAMBIA

* Y ESTO CAMBIA CONTINUAMENTE EL
PROBLEMA CON EL QUE ESTAMOS TRABAJANDO

* LOS AGENTES SE ENFRENTAN A UN
EQUILIBRIO ENTRE LAS REWARDS EN
DIFERENTES PERÍODOS.

== RL EN Python ==

- LA LIBRERÍA MÁS COMÚN PARA EXPERIMENTAR
DE REFORZO ES:
 - Open AI GYM.



Import the library

import gym

Make an environment

env = gym.make("environment-type")

Render (show) the environment

env.render()

#####

Q-LEARNING

CON KERNELS

MODULO 6

#####

== INTRODUCCIÓN ==

* Q-LEARNING:

- ALGORITMO DE (RL) APRENDIZAJE POR
REFUERZO MUY UTILIZADO

* RL SE CARACTERIZA POR CAPACITAR A LOS
AGENTES PARA QUE TOMEN SECUENCIAS
DE DECISIONES MAXIMIZANDO LA SUMA
DE RECOMPENSAS ACUMULADAS.

* ES UN ALGORITMO FUERA DE LA
POLÍTICA QUE BUSCA APRENDER EL VALOR
DE TOMAR UNA ACCIÓN ESPECÍFICA EN
UN ESTADO DETERMINADO Y TIENE COMO
OBJETIVO ENCONTRAR LA POLÍTICA DE

SELECCIÓN DE ACCIONES ÓPTIMO PARA UN AGENTE.

== ¿WHAT IS? ==

* ALGORITMO DE RL BASADO EN VALORES.

* FUNCIONA CON UN AGENTE QUE INTERACTUA CON SU ENTORNO

* EL AGENTE APRENDE Y TOMA DECISIONES DE LAS CONSECUENCIAS DE ESTAS ACCIONES.

* EL OBJETIVO PRINCIPAL DE Q-Learning:

- APRENDER UNA POLÍTICA QUE PERMITE AL AGENTE QUE MEDIDAS TOMAR Y EN QUE CIRCUNSTANCIAS PARA MAXIMIZAR SU RECOMPENSA ACUMULADA EN EL TIEMPO

== Q-VALUE FUNCTION ==

* ESTA FUNCIÓN PROPORCIONA UNA MEDIDA DE LA UTILIDAD ESPERADA DE TOMAR LA ACCIÓN EN EL ESTADO S

* POSTERIOREMENTE SEGUIR LA POLÍTICA ÓPTIMA.

* LOS VALORES Q SE ACTUALIZAN DE FORMA ITERATIVA MEDIANTE:

BELLMAN EQUATION

- RECOMPENSA INMEDIATA
- RECOMPENSAS FUTURAS ESTIMADAS.

BELLMAN EQUATION

• REGLA DE ACTUALIZACIÓN PARA EL VALOR Q :

$Q(s,a) \leftarrow Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$	
s	CURRENT STATE
a	CURRENT ACTION
r	REWARD RECEIVED AFTER TAKING ACTION a
s' (s prime)	STATE RESULTING FROM TAKING ACTION a
a' (a prime)	NEXT ACTION
α (alpha)	LEARNING RATE, CONTROLLING THE NEXT EXTENT TO WHICH NEW INFORMATION OVERRIDES THE OLD
γ (gamma)	DISCOUNT FACTOR, MODELS THE AVERAGE FUTURE REWARDS

Q-LEARNING WITH Keras

1. INICIAR EL ENTORNO Y PARÁMETROS

* DEFINIR UNA PLATAFORMA COMO:

- OPENAI GYM.

* INICIALIZAR LA Q-TABLE.

(Tabla de pares de acciones de estado.)

* ESTABLECE LOS HIPERPARÁMETROS:

- TASA DE APRENDIZAJE (Learning Rate / alpha)

- FACTOR DE DESCUENTO (Discount factor / gamma)

- TASA DE EXPLORACIÓN (Exploration Rate / epsilon)

2. CREAR LA RED Q:

* CREAR Q-NETWORK QUE SE APROXIME A LA FUNCIÓN Q-VALUE:

- ESTA RED Q SUSTITUYE A LA Q-TABLE EN ENTORNOS CON GRANDES ESPACIOS DE ESTADO.

3. ENTRENAR LA Q-NETWORK:

* IMPLEMENTAR CICLO DE ENTRENAMIENTO

- EL AGENTE INTERACTÚA CON EL ENTORNO
- SELECCIONA ACCIONES
- RECIBE RECOMPENSAS
- HAGA LA TRANSICIÓN A NUESTROS ESTADOS
- ACTUALICE LOS Q-VALUES.

④ - EVALUAR EL AGENTE:

* EVALUAR DESPUÉS DEL ENTRENAMIENTO:

- EVALUAR SU DESEMPEÑO
- CAPACIDAD PARA MAXIMIZAR REWARDS

== Step 1: INICIAR EL ENTORNO Y PARÁMETROS ==

* ENTORNO: Cont Pole (OpenAI Gym)

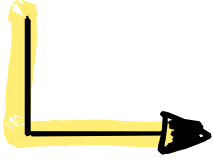
- EL OBJETIVO ES EQUILIBRAR UN PALETE SOBRE UN CARRO.

* PARÁMETROS:

- LEARNING RATE
- DISCOUNT FACTOR
- EXPLORATION RATE

== Step 2: BUILD THE Q-NETWORK ==

* EN EL Q-LEARNING TRADICIONAL SE USA UNA Q-TABLE PARA ALMACENAR LOS Q-VALUES DE TODAS LAS PAIRES DE ACCIONES DE ESTADO.



• PARA ENTORNOS CON ESPACIOS DE ESTADOS GRANDES O CONTINUOS LA Q-TABLE NO SIRVE.



* UTILIZAMOS NUESTRA "Q-NETWORK" PARA APROXIMAR LA "Q-VALUE FUNCTION".

* EN KERAS PODAMOS CONSTRUIR ESTA RED Q USANDO UNAS CUANTAS CAPAS DENSAS.

* EL TAMAÑO DE LA CAPA DE ENTRADA DEBE COINCIDIR CON EL TAMAÑO DEL ESTADO:

$\text{input-layer-size} = \text{state-size}$

* EL TAMAÑO DE OUTPUT LAYER CON EL TAMAÑO DE LA ACCIÓN: $\text{output-layer-size} = \text{action-size}$

* LAS CAPAS OCULTAS PUEDEN TENER CUALQUIER ARQUITECTURA:

- LIBRAMIENTOS:

- 2 a 3 Hidden-layers con
activation = 'relu'.

== Step 3: TRAIN THE Q-NETWORK ==

① * INICIALIZAR EL ESTADO:

- RESTABLECER EL ESTADO PARA
OBTENER EL ESTADO INICIAL

② * SELECCIONAR LA ACCIÓN:

- UTILIZAR LA POLÍTICA "Epsilon".

↳ . PARA COMPUTAR:
- EXPLOTACIÓN.
- EXPLORACIÓN.

• SELECCIONAR UNA DECISION O EXPLORACIÓN
ALEATORIAS

• SELECCIONAR EL Q-VALUE MÁS ALTO PREVIAMENTE.

③ * TOMAR MEDIDAS:

- EJECUTA LA DECISIÓN ELEGIDA EN EL ENTORNO PARA RECIBIR EL SIGUIENTE ESTADO Y LA RECOMPENSA

4. * ACTUALIZAR LOS Q-VALUE:

- UTILIZAR "BELLMAN FUNCTION" PARA ACTUALIZAR LOS VALORES Q.
- CALCULAR EL Q-VALUE OBJETIVO POR EL POR LAS ACCIONES DEL ESTADO ACTUAL
- EXPANDIR LA Q-NETWORK PARA MINIMIZAR LA DIFERENCIA ENTRE EL "Q-VALUE PREDICTED VS Q-VALUE TARGET"

5. * REPEAT:

- CONTINUAR CON EL PROCESO HASTA QUE EL AGENTE ALCANZE UN ESTADO TERMINAL O LOGRE EL OBJETIVO.
- PERKE GRADUAMENTE LA UTILIDAD DE EXPLORACIÓN, EXPLOITACIÓN. POR POR DE EXPLORACIÓN O EXPLOTACIÓN.

Step 4: EVALUATE THIS AGENT

✓ EVALUAR AL AGENTE PERMITIENDO INTERACTUAR CON EL ENTORNO MEDIANTE LA POLICY APRENDIDA

- EL AGENTE DEBE APROVECHAR LOS Q-VALUE APRENDIDOS PARA MÁXIMIZAR LAS RECOMPENSAS.

- EL RENDIMIENTO SE PUEDE MEDIR POR:

- EL TOTAL DE RECOMPENSAS OBTENIDAS EN VARIOS EPISODIOS.

== CODE IMPLEMENTATION ==

Initialize the environment

```
env = gym.make('CartPole-v1')
```

Set hyperparameters

```
alpha = 0.001  —————> Learning Rate  
gamma = 0.99   —————> Discount factor  
epsilon = 1.0   —————> Exploration Rate  
epsilon_min = 0.01  
epsilon_decay = 0.995  
episodes = 100
```

Initialize the Q-table

```
state_size = env.observation_space.shape[0]  
action_size = env.action_space.n  
q_table = np.zeros((state_size, action_size))
```

Define the Q-network

```
def build_q_network(state_size, action_size):
```

```
    model = Sequential()
```

```
    model.add(Dense(24, input_shape=(state_size, ),  
                    activation='relu'))
```

```
model.add(Dense(24, activation='relu'))
```

```
model.add(Dense(action_size, activation='linear'))
```

```
model.compile(loss='mse', optimizer=Adam,
              learning_rate=alpha)
```

```
return model
```

```
# Build the Q-Network
```

```
q_network = build_q_network(state_size, action_size)
```

```
# Train the Q-Network
```

```
episodes = 100
```

```
for episode in range(episodes):
```

```
    state, info = env.reset()
```

```
    state = np.reshape(state, [1, state_size])
```

```
    total_reward = 0
```

```
    for time in range(500):
```

```
        if np.random.rand() <= epsilon:
```

```
            action = np.random.choice(action_size)
```

```
        else:
```

```
            q_values = q_network.predict(state)
```

```
            action = np.argmax(q_values[0])
```



```
next_state, reward, done, trunc, _ = env.step(action)
next_state = np.reshape(next_state, [1, state_size])
total_reward += reward
```

```
if done: reward = -10
```

```
q_target = q_network.predict(state)
```

```
q_target[0][action] = reward + gamma *
```

```
np.amax(q_network.predict(next_state)[0])
```

```
q_network.fit(state, q_target, epochs=1,  
               verbose=0)
```

```
state = next_state
```

```
if done:
```

```
    print(f"Episode: {episode+1} { / episode },  
          Score: {total_rewards}")
```

```
    break
```

```
if epsilon > epsilon_min:
```

```
    epsilon *= epsilon_decay
```

#Evaluate the agent

for episode in range(10):

state, info = env.reset()

state = np.reshape(state, [1, state.size])

total_reward = 0

for time in range(500):

env.render()

q_values = q_network.predict(state)

action = np.argmax(q_values[0])

next_state, reward, done, trunc, _ = env.step(action)

next_state = np.reshape(next_state, [1, state.size])

total_reward += reward

state = next_state

if done:

print(f"Episode: {episode+1} / {episodes},
Score: {total_reward}")

break

env.close()

#####

LABORATÓRIO XII

MODULO 6

#####

IMPLEMENTAÇÃO DE
Q-LEADERS

SEU NEGÓCIO

#####

#####

#####

#####

#####

REDES Q-PROFUNDAS (DQN)

CON KERNELS

MÓDULO 6

#####

== INTRODUCCIÓN ==

DEEP Q-NETWORKS

* Las Deep Q-Networks DQN son una extensión de Q-LEARNING

* UTILIZA LAS REDES NEURONALES PROFUNDAS PARA APROXIMAR LA FUNCIÓN Q-VALUE

* Q-LEARNING TRADICIONAL ES ÚTIL PARA PROBLEMAS CON ESPACIOS ESTADOS PEQUEÑOS DEL CRECIMIENTO EXPONENCIAL DE Q-TABLE

* LOS DQN ABORDAN ESTA LIMITACIÓN MEDIANTE EL USO DE UNA RED NEURONAL PARA ESTIMAR Q-VALUE.

- DEEP Q-NET: UTILIZÓ EL FAMOSO ALGORITMO DQN PARA LOGRAR UN RENDIMIENTO A NIVEL HUMANO AL JUGAR A JUEGOS DE ATARI

* LO INNOVADOR DE LOS DQN ES:

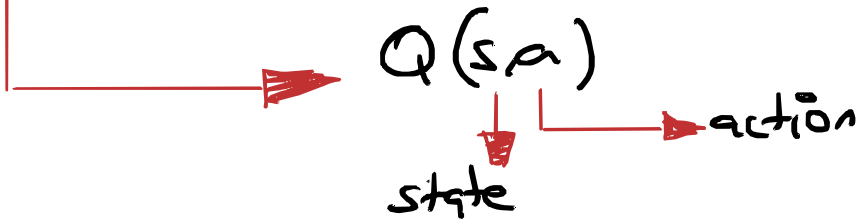
- EL USO DE LA EXPERIENCIA
- LA REPETICIÓN
- LAS REDES OBJETIVO

PARA ESTABLECER EL ENTRENAMIENTO Y MEDIR EL RENDIMIENTO

== CONCEPTOS ==

- SE USA UNA Q-TABLE
- SE USA Q-VALUE FUNCTION:

- Red Neuronal para aproximar la función Q -VALUE ($Q(S,A)$), donde S es el estado y A es la acción



X EXPERIENCE REPLY:

- El STORE AGENT DE DQNS EXPERIMENTA LA RECOMPENSA POR ACCIÓN ESTADO EN EL SIGUIENTE ESTADO EN UN BÚFER DE REPETICIÓN
- DURANTE EL ENTRENAMIENTO, LAS MUESTRAS DE DATOS DE ESTE BÚFER SE UTILIZAN PARA ACTUALIZAR LA RED

X TARGET NETWORK:

- SE UTILIZA PARA GENERAR LOS Q -VALUE DE DESTINO

- PROPORCIONA UNOS OBJETIVO MÁS ESTABLES Y CLAROS LAS ACCIONES DURANTE EL ENTRENAMIENTO.

== STEPS TO == IMPLEMENT DQN

• PASOS SENCILLOS A LOS DE Q-LEARNING :

1. INICIALIZAR EL ENTORNO Y LOS PARÁMETROS

- Define environment like OpenAI's Gym
- Set hyperparameters for training
- Initialize the replay buffer

↓
ALMACENAR LAS EXPERIENCIAS

2. CREAR Q-NETWORK Y TARGET NETWORK

- SE CREAN DOS REDES :

- PRINCIPAL
- OBJETIVO

- TIENEN LA MISMA ARQUITECTURA, PERO
LOS PESOS DE LAS REDES OBJETIVO SE
ACTUALIZAN CON MENOR FRECUENCIA
PARA PROPORCIONAR TARGETS DE
Q-VALUE ESTABLES RESPECTO AL
ENTRENAMIENTO.

3. IMPLEMENTAR LA REPRODUCCIÓN DE EXPERIENCIAS:

(Experience Replay)

- ALMACENAR LAS EXPERIENCIAS RECOR-
DADAS EN UN BUFER DE
REPRODUCCIÓN
- COGER LAS MUESTRAS DE MÍNUTOS
ALEATORIOS DE ESTE BUFER PARA
ALIMENTAR LA Q-NETWORK
- AYUDA A ROMPER LA CORRELACIÓN ENTRE
EXPERIENCIAS CONSECUTIVAS

4. TRAINING THE Q-NETWORK

- DETERMINACIÓN DE Q-VALUES UTILIZANDO
EQUACIÓN BELLMAN

- LA Q-STRUCTURE PRINCIPAL SE
ACTUALIZA UTILIZANDO LOS GEDDIENTES
CALCULADOS A PARTIR DE LA DIFERENCIA
ENTRE LOS Q-VALUE PROMEDIADOS Y LOS
Q-VALUE TARGET OBTENIDOS DE LA RED
OBJETIVO

5. EVALUAR EL AGENTE

- DESPUÉS DEL ENTRENAMIENTO, SE EVALUA EL AGENTE
REEMPLAZANDO INTERACTUANDO CON EL ENTORNO
MEDIANTE LA POLÍTICA APRENDIDA.
- EL RENDIMIENTO DEL AGENTE SE MIDE
POR LA RECOMPENSA PROMEDIADA OBTENIDA
EN VARIOS EPISODIOS

== CODE IMPLEMENTATION ==

initialize the environment

env = gym.make('CartPole-v1')

①

Set hyperparameters

$\alpha = 0.001$ $\longrightarrow$ Learning Rate
 $\gamma = 0.99$ $\longrightarrow$ Discount factor
 $\epsilon = 1.0$ $\longrightarrow$ Exploration Rate
 $\epsilon_{\min} = 0.01$
 $\epsilon_{\text{decay}} = 0.995$
episodes = 100
batch_size = 64
memory_size = 2000

Initialize replay buffer

memory = deque(maxlen = memory_size)

Get state and action sizes

state_size = env.observation_space.shape[0]
action_size = env.action_space.shape[0]

#####

def build_q_network(state_size, action_size)

model = Sequential()

model.add(Dense(24, input_dim=state_size,

model.add(Dense(24, activation='relu',

model.add(Dense(action_size, activation='linear'))

```
model.compile(loss='mse', optimizer=Adam
              (learning_rate = alpha))
return model
```

Build Q-Network and Target Network

```
q_network = build_q_network(state_size, action_size)
target_network = build_q_network(state_size, action_size)
target_network.set_weights(q_network.get_weights())
```

```
##### * #####
#####
```

Experience replay

③

```
def remember(state, action, reward,
             next_state, done):
```

```
    memory.append((state, action, reward, next_state, done))
```

```
def replay(batch_size):
```

```
    minibatch = np.random.choice(len(memory),
```

```
                                batch_size, replace=False)
    for index in minibatch:
```

```
        state, action, reward, next_state, done = memory[index]
```

target = q-network.predict(state)

if done:

target[0][action] = reward

else:

t = target-network.predict(next_state)

target[0][action] = reward + gamma *
np.max(t)

q-network.fit(state, target, epochs=1,
verbose=0)

*

Initialize the environment

④

env = gym.make('CartPole-v1')

Set hyperparameters

alpha = 0.001 → Learning Rate

gamma = 0.99 → Discount factor

epsilon = 1.0 → Exploration Rate

epsilon_min = 0.01

epsilon_decay = 0.995

episodes = 100

batch_size = 64

memory_size = 2000

Initialize replay buffer

memory = deque(maxlen = memory-size)

Get state and action sizes

state_size = env.observation_space.shape[0]

action_size = env.action_space.shape[0]

Main train loop

for episode in range(episodes):

state, info = env.reset()

state = np.reshape(state, [1, state_size])

total_reward = 0

for time in range(500):

if np.random.rand() <= epsilon:

action = np.random.choice(action_size)

else:

q-values = q-network.predict(state, verbose = 0)

action = np.argmax(q-values[0])

next_state, reward, done, truncated, info = env.step(action)

next_state = np.reshape(next_state, [1, state_size])

total_reward += reward

if done:

reward =

remember (state, action, reward, next-state, done)

break

state = next-state

if len(memory) > batch-size:

replay (batch-size)

if epsilon > epsilon-min:

epsilon *= epsilon-decay

#Update the target network weights every
10 episodes

if episode % 10 == 0:

target_network.set_weights (network.
get_weights())

print (f"Episode: {episode+1} / {episodes} , Score:
{total_reward}")

* #####
#####

5

for episode in range(10):

state, info = env.reset()

state = np.reshape(state, [1, state_size])

total_reward = 0

for time in range(500):

env.render()

q_values = q_network.predict(state, verbose=0)

action = np.argmax(q_values[0])

next_state, reward, done, trunc, _ = env.step(action)

next_state = np.reshape(next_state, [1, state_size])

total_reward += reward

state = next_state

if done:

print(f"Episode: {episode+1} / {episodes} {Score: } total_reward { "}

env.close()

#####

LABORATORIO XIII

MODULO 6

#####

CREACIÓN DE UNA Q-NETWORK
PROFUNDA (DON)

CON KETAS

#####

#####

#####

#####

* RESUMEN RL CON VERBAS *



(Aprendizaje de Refuerzo)

- LAS INNOVACIONES CLAVES DE LAS REDES Q PROFUNDAS (DQN) INVOLUCRAN LA REPETICIÓN DE EXPERIENCIAS Y LAS REDES OBJETIVO, QUE DYNAMICA O ESTABILIZAR EL ENTRENAMIENTO Y REDUCIR EL RENDIMIENTO
- RL ES MUY POTENTE PARA ENTRENAR AGENTES A TOMAR DECISIONES EN ENTORNOS COMPLEJOS. Y Q-LEARNING ES UNO DE LOS ALGORITMOS FUNCIONALES DE ESTE CAMPO.
- LA ESTRUCTURA DE Q-LEARNING REQUIERE EN LA FUNCION DE VALOR: $Q(s, a)$
- LOS Q-VALUES SE ACTUALIZAN ITERATIVAMENTE CON BACKUP.

Module 6 Glossary: Introduction to Reinforcement Learning with Keras

Welcome! This alphabetized glossary contains many terms used in this course. Understanding these terms is essential when working in the industry, participating in user groups, and participating in other certificate programs.

Term	Definition
alpha	The learning rate that determines how much newly acquired information influences the update of the current value or policy.
Bellman equation	A necessary condition for optimality associated with the mathematical optimization method known as dynamic programming. It is named after named after Richard E. Bellman.
Deep Q-networks (DQNs)	An extension of Q-learning that uses deep neural networks to approximate the Q-value function. It addresses the limitation using a neural network to estimate the Q-values, allowing the algorithm to scale to environments with large or continuous state spaces.
epsilon	The exploration rate that controls the probability of choosing a random action instead of the best-known action to encourage exploration of the environment.
gamma	The discount factor determines the importance of future rewards relative to immediate rewards in the value function.
Hyperparameters	The settings in machine learning models that are set before training and control the learning process, such as learning rate, batch size, and the number of layers.
OpenAI's Gym	An open-source Python toolkit that provides developers with a simulated environment to develop and test reinforcement learning agents for deep learning models.
Q-learning	An off-policy algorithm that seeks to learn the value of taking a specific action in a given state and aims to find the optimal action-selection policy for an agent.
Q-network	A neural network used to approximate the Q-value function, mapping state-action pairs to their expected future rewards.
Q-table	A lookup table where each entry estimates the cumulative reward obtained by taking a given action in a given state and following the optimal policy afterward.
Q-value	A function that estimates the expected future rewards for taking a specific action in a given state and following the optimal policy thereafter.
Reinforcement learning	A powerful paradigm in machine learning that focuses on training agents to make sequences of decisions by maximizing a notion of cumulative reward.
Supervised learning	A category of machine learning technology that uses labeled data sets to train algorithms to predict outcomes and recognize patterns.
Unsupervised learning	A category of machine learning technology that uses algorithms to analyze and cluster unlabeled data sets.



Skills Network

Practice Project Overview: Fruit Classification Using Transfer Learning

Estimated Reading Time: 15 minutes

Project overview

In this practice project, you will learn to classify images of fruits into different categories using transfer learning. The pre-trained VGG16 model will be used as the base, and custom layers will be added to fine-tune the model for our specific task. This approach allows us to leverage the knowledge learned from a large dataset (ImageNet) and apply it to a smaller, custom dataset with minimal computational resources and data.

Aim

The aim is to classify images of different fruits using a machine learning model. You will train a convolutional neural network (CNN) to recognize various fruit categories using the power of transfer learning with a pre-trained model.

Final Output

A trained model capable of classifying fruit images into specified categories with good accuracy.

Learning objectives

After completing this project, you will be able to:

- Prepare data into a directory structure
- Create sample images
- Verify the directory structure and images
- Set up the data generator
- Build and compile the model
- Train and evaluate the model using callbacks
- Plot and visualize the model

Prerequisites

- Basic knowledge of Python and Keras
- TensorFlow installed in your Python environment
- A dataset of fruit images organized in subdirectories for each class

Instructions

You need to perform the following tasks to complete the project after preparing the data:

- Task 1: Print the version of TensorFlow
- Task 2: Create a test generator using the test_datagen object
- Task 3: Print the length of the train generator
- Task 4: Print the summary of the model
- Task 5: Compile the model
- Task 6: Plot accuracy curves for training and validation sets
- Task 7: Plot loss curves for training and validation sets (fine-tuned model)
- Task 8: Plot accuracy curves for training and validation sets (fine-tuned model)
- Task 9: Plot a test image using the Extract Features Model
- Task 10: Plot a test image using the Fine-Tuned Model

Key Takeaways

- Transfer learning enables leveraging pre-trained models for custom tasks with limited data.
- Data augmentation helps improve model generalization on small datasets.

Let's start this guided project and train a convolutional neural network (CNN) to recognize various fruit categories using the power of transfer learning with a pre-trained model.



Skills Network

Final Project Overview: Classify Waste Products Using Transfer Learning

Estimated Reading Time: 15 minutes

Project Scenario

The city of GreenCity has been struggling with waste management, especially in distinguishing between recyclable and organic waste. The local waste management organization, EcoClean, is tasked with improving the efficiency of waste sorting. However, the current manual process is both time-consuming and error-prone. To enhance the system, EcoClean wants you to develop an AI-powered solution that can automatically classify waste products using image recognition techniques. This project aims to build a model that can differentiate between recyclable and organic waste products using transfer learning.

Project Background

EcoClean currently lacks an efficient and scalable method to automate the waste sorting process. The manual sorting of waste is not only labor-intensive but also prone to errors, leading to contamination of recyclable materials. The goal of this project is to leverage machine learning and computer vision to automate the classification of waste products, improving efficiency and reducing contamination rates. The project will use transfer learning with a pre-trained VGG16 model to classify images.

Aim

The aim of the project is to develop an automated waste classification model that can accurately differentiate between recyclable and organic waste based on images. By the end of this project, you will have trained, fine-tuned, and evaluated a model using transfer learning, which can then be applied to real-world waste management processes.

Final Output: A trained model that classifies waste images into recyclable and organic categories.

Learning objectives

After completing this project, you will be able to:

- Apply transfer learning using the VGG16 model for image classification
- Prepare and preprocess image data for a machine learning task
- Fine-tune a pre-trained model to improve classification accuracy
- Evaluate the model's performance using appropriate metrics
- Visualize model predictions on test data

Instructions

To achieve the above objectives, you will complete the following tasks:

Task 1: Print the version of TensorFlow

Task 2: Create a `test_generator` using the `test_datagen` object

Task 3: Print the length of the `train_generator`

Task 4: Print the summary of the model

Task 5: Compile the model

Task 6: Plot accuracy curves for training and validation sets (`extract_feat_model`)

Task 7: Plot loss curves for training and validation sets (`fine-tune model`)

Task 8: Plot accuracy curves for training and validation sets (`fine-tune model`)

Task 9: Plot a test image using Extract Features Model (`index_to_plot = 1`)

Task 10: Plot a test image using Fine-Tuned Model (`index_to_plot = 1`)

Let's start the project by developing and deploying a trained model that classifies waste images into recyclable and organic categories.



Skills Network